BUSINESS CATALOG

Project Handbook

A practical guide to the storefront, staff tools, Supabase database, setup, operations, deployment, and readiness checks.

Prepared from the project files in this repository
Documentation edition · June 2026

Contents

1. Project at a glance
2. How the application is organized
3. Store and staff features
4. Supabase data model and security
5. Initial setup and SQL installation
6. Configuration and deployment
7. Routine operation and testing
8. Current limitations and launch checklist

> **Important:** This handbook documents the implementation currently present in the repository. It does not claim that SQL has been run on a live Supabase project, that n8n is configured, or that a payment gateway is connected.

1. Project at a glance

Business Catalog is a mobile-first online catalogue and ordering application for cosmetics, toiletries, and wholesale supply. It is designed around the Nigerian Naira (NGN) and delivery zones configured in Supabase.

Area	Current implementation
Frontend	Multiple standalone HTML pages with page-level CSS and vanilla JavaScript; no frontend framework or package manifest is present.
Backend services	Supabase Auth, Postgres with Row Level Security (RLS), and Supabase Storage via the Supabase JavaScript v2 CDN.
Shared browser code	`backend.js` loads live catalogue and customer/staff data and performs role-aware initialization. `supabase-client.js` initializes the client; `auth.js` provides auth helpers.
Deployment	Netlify configuration exists. The site can also be hosted as static files, but the older GitHub Pages notes do not match the current root-level file structure in every detail.
Automation	An invoice upload page can POST to an optional n8n webhook. The webhook setting is blank and the workflow itself is intentionally not included.

Repository map

Customer pages: `index.html`, `shop.html`, `product-details.html`, `cart.html`, `checkout.html`, `payment.html`, `receipt.html`, `my-orders.html`, `order-details.html`, `my-quotes.html`, `supply-request.html`, `wholesale.html`, `wishlist.html`, `chat.html`, `notifications.html`, `support.html`, `account.html`, `cancel-order.html`, `signin.html`, `register.html`, `404.html`.

Staff pages: `staff-login.html`, `admin-dashboard.html`, `admin-products.html`, `admin-edit-product.html`, `admin-orders.html`, `admin-order-details.html`, `admin-invoice.html`, `admin-invoice-ai.html`, `admin-supply-requests.html`, `admin-supply-quote.html`, `admin-customers.html`, `admin-customer-details.html`, `admin-credit-ledger.html`, `admin-inbox.html`, `admin-reports.html`, `admin-categories.html`, `admin-delivery-zones.html`, `admin-staff.html`, `admin-activity-log.html`.

Shared/runtime files: `config.js`, `supabase-client.js`, `auth.js`, `backend.js`, `inject-env.js`, and `netlify.toml`.

2. How the application is organized

1. A browser opens a static HTML page from the site host.
2. The page loads Supabase JS v2, the public browser configuration, and shared client/auth/backend code.
3. The browser uses the signed-in Supabase session for customer or staff requests. Public catalogue reads use the public client.
4. Postgres RLS policies enforce which rows the signed-in user may read or change. Key operations such as placing orders and changing credit balances use database functions.
5. Private uploads are stored in Supabase Storage and accessed using authenticated policies or short-lived signed URLs.
6. When a webhook URL is configured, the invoice screen sends invoice metadata and a short-lived file URL to n8n.

Shared files

File	Purpose
`config.js`	Browser-visible Supabase URL, publishable/anon key, and currently blank n8n webhook URL.
`supabase-client.js`	Creates and exposes the Supabase browser client as `BCLive.sb`.
`auth.js`	Signup, sign-in, sign-out, and staff access helpers.
`backend.js`	Fetches catalogue and account data, maps records for pages, and initializes customer/staff pages.
`inject-env.js`	Netlify build helper; requires Supabase URL and public key environment variables and writes generated public configuration.

> **Browser configuration is public.** Anything in HTML or JavaScript shipped to the browser can be inspected. A Supabase publishable/anon key is intended for public clients and must be protected by correct RLS. Never place a Supabase service-role key, database password, or private n8n credential in `config.js`.

3. Store and staff features

Customer experience

- Browse and search products, view richer product information, availability, and configured wholesale tiers.
- Maintain a cart and submit an order after signing in; database logic validates stock and recalculates the price.
- Review orders, order details, receipts, and submit a cancellation or return request.
- Save products, manage the account, request wholesale quotations, accept an eligible quote, and view notifications or support messages.

Staff experience

- Manage products, categories, delivery zones, orders, customer records, reports, staff, and activity.
- Review supply requests and create quotations; accepted, priced quotes can be converted to orders.
- View and update a customer credit account through staff-only database functions with a ledger trail.
- Upload supplier invoices to private storage, inspect their status, and optionally dispatch them to n8n for extraction.

Checkout and payment status

The checkout supports the stored payment-method values `online`, `cash`, and `transfer`. The order RPC creates an order with payment status `pending`. A payment gateway, card charging flow, and automated payment verification are not implemented in the inspected checkout. Do not treat selecting “online” as proof that a payment was collected.

4. Supabase data model and security

Data area	Tables / records	Purpose
People	`profiles`, `addresses`	Auth-linked profile, customer/business details, role and tier.
Catalogue	`categories`, `products`, `product_tiers`, `delivery_zones`	Products, inventory, product details, bulk prices, and delivery fees.
Orders	`orders`, `order_items`, `order_requests`	Order totals, line items, and cancellation/return requests.
Wholesale	`supply_requests`, `supply_request_items`, `supply_request_attachments`	Customer requests, staff quotations, quote conversion, and supporting files.
Engagement	`wishlist`, `chat_threads`, `messages`, `notifications`	Saved products, conversations, and customer notifications.
Staff operations	`activity_log`, `settings`, `customer_notes`	Administrative records, settings, and internal customer notes.
Credit and invoices	`customer_credit_accounts`, `customer_credit_ledger`, `supplier_invoices`	Staff-managed credit balances and supplier invoice extraction records.

Important database functions

- `place_order(...)` validates items and stock, reads current prices and wholesale tiers, computes delivery, decrements stock, and creates the order.
- `submit_supply_request(...)` validates a signed-in customer's business request and its line items.
- `convert_supply_quote_to_order(...)` converts the signed-in customer's eligible quote to an order once and stores the resulting order ID.
- `staff_manage_customer_credit(...)` creates or updates a credit limit and enabled state after checking staff access.
- `staff_adjust_credit_balance(...)` changes a credit balance only for staff, checks the limit/balance, and writes a ledger entry.

RLS is enabled in the base schema. Catalogue data is readable publicly where allowed; customers generally access their own records; staff policies use `is_staff()` to allow staff/owner access. Private invoice and request files use separate buckets and Storage policies.

> **Owner setup:** After registering and confirming the account, an authorized project owner must promote the first staff account using the profile update instruction in the SQL comments. Protect this access carefully; never promote an account based only on an unverified email supplied by a user.

5. Initial setup and SQL installation

For a brand-new Supabase project

1. Create a Supabase project and keep its database password private.
2. In Supabase, open **SQL Editor** and run `supabase-schema.txt` once to create the base tables, policies, functions, image bucket, and sample catalogue.
3. Run `supabase-fixes.sql` to apply the additive profile/product/chat and checkout updates.
4. Run `supabase-remaining-features.sql` to add credit ledger, private invoice/request uploads, quote conversion, and related policies/functions.
5. Review the SQL Editor results for errors before proceeding. Verify that the expected tables, functions, and Storage buckets exist.
6. Set the site's Supabase URL and publishable/anon key in the browser configuration or deployment environment.
7. Register a customer account, confirm email if confirmation is enabled, and promote the intended first owner using a trusted admin-controlled SQL session.
8. Sign in as that owner and verify the staff area. Test as a separate customer account as well.

For an existing project

Do not rerun the fresh-install base schema over populated production data. Back up the database, inspect the current schema, then apply only the missing migrations in order: `supabase-fixes.sql`, followed by `supabase-remaining-features.sql`. Confirm that the starting schema is compatible and review each migration result.

> **SQL source of truth:** The repository's three SQL files are separate steps, not interchangeable alternatives. The base schema contains sample records and initial policies. Never run SQL copied from an outdated handover document instead of checking the current files.

Authentication settings

In Supabase Auth, configure the site URL and allowed redirect URLs for the exact local and production origins you use. Decide whether email confirmation is required, configure SMTP for real customer mail, and test password recovery/confirmation flows. Do not assume a redirect works until tested on the published hostname.

6. Configuration and deployment

Local/browser configuration

The current `config.js` defines `SUPABASE_URL`, `SUPABASE_ANON_KEY`, and an intentionally empty `N8N_INVOICE_WEBHOOK_URL`. Use the project's Supabase publishable (or legacy anon) key only. Keep the Supabase service-role key and all server credentials out of the repository and browser.

Netlify

`netlify.toml` runs `node inject-env.js` and publishes the repository root. Add the expected Supabase URL and public-key environment variables in the Netlify site settings. The current injection script writes Supabase public config but does not preserve or inject the n8n webhook URL. A build may replace a manually edited config file; verify the generated deployment config before testing. For production invoice automation, prefer a secured server-side relay instead of exposing an n8n endpoint in frontend code.

GitHub Pages or another static host

The pages are root-level static files and do not require a frontend bundle. Configure the public Supabase values in `config.js`, publish the correct root folder, and test all relative links at the actual repository subpath/hostname. Static hosting does not make browser configuration private. The included `GITHUB_PAGES.txt` and `HANDOVER_PROMPT.txt` describe an older layout in places; use the current repository file list and scripts as the source of truth.

7. Routine operation and testing

Recommended test sequence

1. Open the deployed store over HTTPS and confirm that the catalogue loads from Supabase without console/network errors.
2. Register a new retail customer and a business customer; verify profile creation and email-confirmation behavior.
3. As a normal customer, verify that staff pages redirect or deny access and that customer records are private to their owner.
4. As staff, update a product, price, stock, and delivery-zone setting; verify the storefront reflects the changes.
5. Place a low-value test order. Confirm that the stored order total is calculated from database prices, stock decrements once, and the customer can see only their own order.
6. Test a quote request, staff quote, customer acceptance/conversion, and the resulting order link.
7. Test a credit-account adjustment and verify the limit rules and ledger record. Use test accounts only.
8. Upload a non-sensitive test invoice and verify private access, queued status, and clean removal/retention behavior.
9. Review Supabase logs, RLS policies, Auth redirect settings, Storage policies, and production error handling before launch.

Useful checks before pushing

- Review `git status` and the complete diff; avoid committing local credentials or generated private data.
- Run `git diff --check` and a link/reference check for the static HTML pages.
- Open key mobile and desktop pages in a browser and test sign-in, checkout, admin navigation, and form error states.
- Run migrations against a disposable/staging Supabase project first. The current code repository does not include an automated migration/test harness.

8. Current limitations and launch checklist

Capability	Current status	Before production
Supabase SQL	SQL scripts are present; execution on the user's Supabase instance has not been verified by this documentation task.	Run the appropriate migration sequence in staging, review errors, back up, then apply to production.
Online payments	Payment method is recorded; no gateway charge/confirmation flow is established in the inspected checkout.	Integrate a payment provider with server-side verification and idempotent order/payment state updates.
n8n	Invoice UI and outgoing webhook contract exist; URL is blank and no workflow is created.	Build/test the workflow and secure the public-to-private boundary before processing real invoices.
Email, redirects	Dependent on Supabase Auth and hosting configuration.	Configure a production mail provider, URLs, and end-to-end account recovery/confirmation tests.
Deployment config	Netlify environment injection is present; older handover/deployment notes may be stale.	Test the exact build and published environment, including generated `config.js`.

Go-live checklist

- Database Migrations applied, policies reviewed, owner role assigned securely, backup/restore plan tested.
- Security No service-role key in the browser or repository; RLS and Storage access tested with customer and staff accounts.
- Commerce Real prices, inventory, zones, and order notifications confirmed; payment methods accurately explained to customers.
- Automation n8n workflow tested using synthetic invoices; no production webhook secret exposed in frontend files.
- Operations Production hostname, HTTPS, Auth redirects, email delivery, support contact, and recovery procedures checked.

This handbook is an implementation guide, not a penetration test, financial audit, or guarantee that external services are configured. Recheck the live project before accepting real customer data or payments.